PROJET BASES DE DONNÉES

Modélisation MERISE · Oracle 23ai · PL/SQL · Optimisation

Gestion d'une Constellation de Nano-Satellites

CubeSat Earth Observation System — NanoOrbit

Niveau 2ème année cycle ingénieur	Groupes Binômes ou trinômes	Module ALTN83 — BD Réparties

1. Contexte et présentation du projet

La startup NanoOrbit exploite une constellation de CubeSats pour surveiller des zones climatiques sensibles : déforestation, fonte des glaces, qualité de l'air, évolution du trait de côte. Pour piloter efficacement cette constellation, elle a besoin d'un système d'information robuste, capable de :

- Gérer une flotte de satellites miniaturisés (CubeSats 1U à 12U)
- Planifier et tracer les fenêtres de communication avec les stations au sol
- Administrer les missions scientifiques et les instruments embarqués
- Garantir la cohérence des données par des mécanismes actifs (triggers, procédures)
- Préparer une architecture de données distribuée par zone géographique

Ce projet constitue le projet de validation du module ALTN83. Il est organisé en quatre phases progressives qui couvrent l'ensemble du syllabus : modélisation, implémentation Oracle, programmation PL/SQL et exploitation avancée.

1.1 Périmètre fonctionnel

Le système d'information à modéliser couvre six domaines fonctionnels :

- La gestion de la flotte de satellites : caractéristiques techniques, statut opérationnel
- La gestion des orbites : paramètres orbitaux, type d'orbite, couverture géographique
- La gestion des instruments embarqués : capteurs, caméras, équipements scientifiques
- La gestion des stations au sol : localisation, capacités de communication
- La planification des fenêtres de communication : créneaux de passage, volumes échangés
- La gestion des missions : objectifs, zones cibles, participation des satellites

1.2 Architecture cible

NanoOrbit opère depuis trois centres de contrôle géographiquement distincts : **Paris** (Europe), **Singapour** (Asie-Pacifique) et **Houston** (Amériques).

Chaque centre gère ses propres stations au sol et fenêtres de communication, mais partage un référentiel global de satellites, d'orbites et de missions.

Cette architecture distribuée sera l'objet de la réflexion en Phase 1 et constituera le fil directeur des phases suivantes.

2. Niveaux de réalisation

Le projet est conçu pour être accessible à tous les groupes tout en permettant aux plus avancés d'aller plus loin. Deux niveaux de réalisation sont définis.

Niveau 1 — Obligatoire (tous les groupes)	Niveau 2 — Bonus (groupes avancés)
• Phase 1 complète (MCD, MLD, répartition) • Phase 2 : DDL complet + 3 triggers obligatoires • Phase 3 : Paliers 1 à 5 (16 exercices — blocs, curseurs, 3 procédures, 2 fonctions) • Phase 4 : Vues V1–V3 + CTE simple + sous-requêtes	• Phase 2 : 2 triggers supplémentaires (bonus) • Phase 3 : Package pkg_nanoOrbit complet (SPEC + BODY + scénario) • Phase 4 : Vue matérialisée + fonctions analytiques + MERGE INTO + EXPLAIN PLAN • Rapport de pilotage intégral (exercice de synthèse)

⚠ A propos du MLD de référence

Le MLD de référence NanoOrbit (10 tables) ne vous est pas fourni à l'avance. Vous devez le construire vous-même en Phase 1 à partir du cahier des charges et des règles de gestion.

Il vous sera communiqué par votre chargé de TD uniquement après validation de votre MCD — il servira alors de base commune pour les phases 2, 3 et 4.

Les variantes mineures justifiées dans vos scripts restent acceptées.

PHASE 1
Conception & Architecture distribuée

Modélisation MERISE · MCD · MLD · Répartition

Durée indicative 3 h 30	Prérequis Modélisation MERISE (S6)	SGBD — (modélisation uniquement)

1.1 Objectifs pédagogiques

- Analyser un cahier des charges et classifier les règles de gestion (structure / contrainte / procédural)
- Construire un MCD conforme à la méthode MERISE en gérant les associations porteuses d'attributs
- Dériver le MLD relationnel en appliquant les règles de passage MERISE
- Identifier les données locales et globales dans une architecture distribuée multi-sites
- Anticiper les risques de cohérence dans un système réparti

1.2 Étape 1 — Dictionnaire des données et classification

Répertoriez l'ensemble des attributs du système en vous appuyant sur le contexte décrit en section 1. Pour chaque attribut, précisez :

Attribut / Code	Type Oracle	Obligatoire	Unique	Contraintes / Remarques
id_satellite	VARCHAR2(20)	OUI	OUI	Code alphanumérique, immuable (ex: SAT-001)
statut_actuel	VARCHAR2(30)	OUI	NON	Opérationnel / En veille / Défaillant / Désorbité
… (à compléter)	…	…	…	

Classez ensuite chaque règle de gestion dans l'une des trois catégories suivantes :

Catégorie de garantie	Exemples de règles NanoOrbit
Structure relationnelle (PK, FK, UNIQUE)	Identifiant satellite unique (RG-S01), unicité altitude+inclinaison (RG-O02)
Contrainte simple (CHECK, NOT NULL)	Durée fenêtre entre 1 et 900 s (RG-F04), date_fin nullable (RG-M01)
Mécanisme procédural (Trigger / Procédure)	Chevauchement de fenêtres (RG-F02/F03), satellite désorbité bloqué (RG-S06)

1.3 Étape 2 — Modèle Conceptuel de Données

Produisez un MCD respectant les conventions MERISE et couvrant l'ensemble des règles de gestion. Portez une attention particulière aux points suivants :

- L'entité-association EMBARQUEMENT entre SATELLITE et INSTRUMENT, porteuse des attributs date_integration et etat_fonctionnement (RG-S04)
- L'entité-association PARTICIPATION entre SATELLITE et MISSION, porteuse de l'attribut role_satellite (RG-M03)
- La relation impliquant FENETRE_COM, SATELLITE et STATION_SOL : binaire ou ternaire ? Justifiez votre choix
- L'intégration de CENTRE_CONTROLE dans le modèle et sa relation avec STATION_SOL

> ⚠ **Point de vigilance**
> Certaines contraintes ne sont pas exprimables dans le MCD (chevauchement de fenêtres, satellite désorbité bloqué). Identifiez-les et signalez-les dans votre note de modélisation — elles seront implémentées par des triggers en Phase 2.

1.4 Étape 3 — Modèle Logique de Données

Traduisez votre MCD en modèle relationnel. Indiquez pour chaque table : les clés primaires soulignées, les clés étrangères avec la notation #FK, et les types Oracle cibles (VARCHAR2, NUMBER, DATE, TIMESTAMP). Vérifiez la troisième forme normale (3NF) et identifiez les colonnes candidates à l'indexation.

1.5 Étape 4 — Réflexion sur l'architecture distribuée

NanoOrbit opère depuis trois centres de contrôle (Paris, Singapour, Houston). En vous appuyant sur votre MLD, répondez aux questions suivantes :

Q1	**Quelles tables du MLD sont strictement locales à un centre de contrôle ?** Justifiez en expliquant pourquoi ces données n'ont pas vocation à être partagées entre centres.
Q2	**Quelles tables doivent être globales (accessibles depuis tous les centres) ?** Quels mécanismes de synchronisation proposez-vous (réplication, partage en lecture seule, etc.) ?
Q3	**Comment le centre de Singapour peut-il continuer à planifier des fenêtres de communication si le serveur central est indisponible ?** Proposez une architecture de fragmentation horizontale ou verticale adaptée.
Q4	**Quels risques de cohérence identifiez-vous dans ce système multi-sites ?** Citez deux scénarios concrets (ex : mise à jour simultanée du statut d'un satellite depuis deux centres).

1.6 Livrables Phase 1

Réf.	Livrable	Contenu attendu
L1-A	**Dictionnaire des données**	Tableau complet avec types Oracle, contraintes et classification des règles de gestion
L1-B	**MCD**	Schéma conforme MERISE, entités-associations porteuses identifiées, légende claire
L1-C	**MLD**	Notation textuelle ou schéma graphique, PK soulignées, FK annotées, types Oracle
L1-D	**Note de modélisation**	1 page : justification des 3 choix délicats + réponses aux questions Q1–Q4 sur la répartition

PHASE 2
Schéma Oracle & Triggers

DDL · DML · PL/SQL Triggers · Oracle 23ai

Durée indicative 3 h 30	Prérequis Phase 1 terminée, MLD validé	SGBD Oracle 23ai — NANOORBIT_ADMIN

🎯 **Prérequis de connexion**
Connectez-vous en tant que NANOORBIT_ADMIN sur FREEPDB1 avant de commencer. Vérifiez avec SHOW USER; Toutes les tables sont créées dans ce schéma.

2.1 Objectifs pédagogiques

- Créer les 10 tables du schéma NanoOrbit avec toutes leurs contraintes Oracle
- Insérer un jeu de données de test cohérent dans le respect de l'ordre référentiel
- Implémenter les 5 triggers métier couvrant les règles de gestion non exprimables en DDL
- Maîtriser BEFORE / AFTER, :NEW, :OLD, RAISE_APPLICATION_ERROR et RETURNING … INTO

2.2 Partie 1 — Création du schéma (DDL)

Créez les 10 tables du système NanoOrbit dans l'ordre strict imposé par les dépendances de clés étrangères. L'ordre de création doit être justifié en commentaire dans votre script.

#	Table	Dépend de	Contenu
1	**ORBITE**	—	Référentiel des plans orbitaux
2	**SATELLITE**	ORBITE	Parc de CubeSats
3	**INSTRUMENT**	—	Catalogue des instruments
4	**EMBARQUEMENT**	SATELLITE, INSTRUMENT	Instruments montés sur satellites
5	**CENTRE_CONTROLE**	—	Centres d'opération NanoOrbit
6	**STATION_SOL**	—	Stations d'antenne mondiales
7	**AFFECTATION_STATION**	CENTRE_CONTROLE, STATION_SOL	Rattachement station ↔ centre
8	**MISSION**	—	Missions scientifiques
9	**FENETRE_COM**	SATELLITE, STATION_SOL	Créneaux de communication
10	**PARTICIPATION**	SATELLITE, MISSION	Rôles des satellites dans les missions

Pour chaque table, respectez impérativement les exigences suivantes :

Réf.	Exigence	Règle associée
E1	Tous les attributs NOT NULL sauf date_fin (MISSION) et volume_donnees (FENETRE_COM)	RG-M01, RG-F05
E2	Les champs statut et etat_fonctionnement implémentés en ENUM via CHECK ou domaine Oracle	Dictionnaire des données
E3	Contrainte UNIQUE sur le couple (altitude, inclinaison) dans ORBITE	RG-O02
E4	Contrainte CHECK sur duree dans FENETRE_COM : BETWEEN 1 AND 900	RG-F04
E5	Toutes les FK déclarées avec ON DELETE RESTRICT (comportement par défaut Oracle)	Intégrité référentielle
E6	PK composites dans EMBARQUEMENT et PARTICIPATION	MLD de référence

Questions de réflexion (à répondre en commentaire dans le script)

- Q1 — Pourquoi ne peut-on pas créer SATELLITE avant ORBITE ? Quelle règle de gestion cela traduit-il ?
- Q2 — La règle RG-S06 (satellite désorbité : plus de fenêtre ni de mission) peut-elle être vérifiée au niveau DDL seul ? Quelle solution proposez-vous ?
- Q3 — Comment implémentez-vous la contrainte RG-F02 (pas de chevauchement de fenêtres pour un même satellite) ? Cette contrainte est-elle directement exprimable en CHECK ?
- Q4 — Quel type Oracle choisissez-vous pour format_cubesat (valeurs possibles : 1U, 3U, 6U, 12U) ? Justifiez.

2.3 Partie 2 — Jeu de données initial (DML)

Insérez les données dans l'ordre strict des dépendances. Respectez les valeurs fournies — elles seront réutilisées dans les phases 3 et 4. Un COMMIT; final valide l'ensemble.

Jeu de données à insérer

3 orbites (SSO × 2, LEO × 1) · 5 satellites (dont 1 Désorbité) · 4 instruments · 7 embarquements · 2 centres de contrôle · 3 stations au sol · 3 missions (Active × 2, Terminée × 1) · 7 participations · 5 fenêtres de communication (Réalisée × 3, Planifiée × 2)

Les données exactes à insérer sont celles du MLD de référence (section 2) complétées par les tables CENTRE_CONTROLE et AFFECTATION_STATION. Adaptez les valeurs si votre schéma présente des variantes de nommage.

2.4 Partie 3 — Les 5 triggers métier NanoOrbit

Ces triggers implémentent les règles métier qui ne peuvent pas être exprimées par des contraintes statiques. Activez la sortie console avant de commencer : SET SERVEROUTPUT ON;

Trigger	Niveau	Événement	Règle métier	Règles
T1 — trg_valider_fenetre	**1**	BEFORE INSERT ON FENETRE_COM	Bloque si satellite désorbité ou station en maintenance	RG-S06, RG-G03
T2 — trg_no_chevauchement	**1**	BEFORE INSERT OR UPDATE ON FENETRE_COM	Vérifie l'absence de chevauchement temporel pour le satellite et la station	RG-F02, RG-F03
T3 — trg_volume_realise	**1**	BEFORE INSERT OR UPDATE ON FENETRE_COM	Force volume_donnees à NULL si statut différent de Réalisée	RG-F05
T4 — trg_mission_terminee	**2**	BEFORE INSERT ON PARTICIPATION	Bloque l'ajout d'un satellite si la mission est Terminée	RG-M04
T5 — trg_historique_statut	**2**	AFTER UPDATE OF statut ON SATELLITE	Trace tout changement de statut dans HISTORIQUE_STATUT	RG-S06 (traçabilité)

ℹ Note sur T5 — HISTORIQUE_STATUT
Ce trigger nécessite la création préalable d'une table HISTORIQUE_STATUT (id_historique AI, id_satellite, ancien_statut, nouveau_statut, date_changement TIMESTAMP, motif VARCHAR2). Créez-la après SATELLITE dans l'ordre DDL.

Pour chaque trigger, fournissez : le code PL/SQL complet avec SHOW ERRORS, les cas de test (cas valide + cas en erreur), et le résultat attendu en commentaire.

2.5 Livrables Phase 2

Réf.	Livrable	Contenu attendu
L2-A	**Script DDL complet**	CREATE TABLE × 11 (10 + HISTORIQUE_STATUT), contraintes, commentaires justificatifs Q1–Q4
L2-B	**Script DML**	INSERT INTO pour les 10 tables dans le bon ordre, COMMIT final
L2-C	**Script Triggers**	5 triggers avec tests commentés et résultats attendus
L2-D	**Script de contrôle**	Requêtes de vérification du schéma (user_tables, user_constraints, user_triggers)

PHASE 3
PL/SQL & Package pkg_nanoOrbit

Blocs anonymes · Curseurs · Procédures · Fonctions · Package

Durée indicative 3 h 30	Prérequis Phase 2 terminée, triggers opérationnels	SGBD Oracle 23ai — SET SERVEROUTPUT ON

3.1 Objectifs pédagogiques

- Écrire et tester des blocs PL/SQL anonymes sur le schéma NanoOrbit
- Maîtriser %TYPE, %ROWTYPE, SELECT INTO, IF/ELSIF/CASE, boucles et curseurs
- Créer des procédures et fonctions standalone avec gestion des exceptions
- Assembler le package pkg_nanoOrbit (SPEC + BODY) encapsulant les opérations métier

3.2 Progression pédagogique

Palier	Thème	Ce que vous saurez faire
1	**Bloc anonyme**	DECLARE / BEGIN / END, DBMS_OUTPUT, comptage des satellites et fenêtres
2	**Variables & types**	%TYPE, %ROWTYPE, SELECT INTO, NVL sur les données NanoOrbit
3	**Structures de contrôle**	IF/ELSIF/CASE sur statuts satellite, boucles FOR sur fenêtres
4	**Curseurs**	Cursor FOR Loop, curseur paramétré par mission, OPEN/FETCH/CLOSE
5	**Procédures & fonctions**	Sous-programmes standalone IN/OUT, RAISE_APPLICATION_ERROR
6	**Package pkg_nanoOrbit**	SPEC + BODY, types publics, constantes métier, 7 sous-programmes

3.3 Paliers 1 à 5 — Exercices progressifs

Pour chacun des paliers 1 à 5, rédigez et testez les blocs PL/SQL demandés. Chaque exercice est accompagné du résultat attendu en commentaire. La progression suit exactement la structure des paliers PL/SQL du module.

Palier 1 — Bloc anonyme

- Ex. 1 : Afficher un message de bienvenue et le nombre de satellites, stations et missions de la base
- Ex. 2 : Utiliser SELECT INTO pour récupérer et afficher les caractéristiques du satellite SAT-001

Palier 2 — Variables et types

- Ex. 3 : Lire une ligne complète de SATELLITE avec %ROWTYPE et afficher son statut et sa capacité batterie
- Ex. 4 : Gérer les NULL avec NVL — afficher la résolution d'un instrument (N/A pour les capteurs AIS sans résolution)

Palier 3 — Structures de contrôle

- Ex. 5 : IF/ELSIF — Catégoriser un satellite selon son statut et sa durée de vie restante estimée
- Ex. 6 : CASE — Afficher le type d'orbite d'un satellite et calculer sa vitesse orbitale approximative ($v = 2\pi \times (6371 + \text{altitude}) / \text{période}$)
- Ex. 7 : Boucle FOR — Afficher la grille des volumes de données attendus pour des passages de 5 à 15 minutes avec le débit de la station GS-TLS-01

Palier 4 — Curseurs

- Ex. 8 : SQL%ROWCOUNT — Mettre à jour le statut de plusieurs satellites et afficher le nombre de lignes modifiées
- Ex. 9 : Cursor FOR Loop — Lister tous les satellites avec leur orbite, leur statut et leurs instruments embarqués
- Ex. 10 : Curseur explicite OPEN/FETCH/CLOSE — Satellites opérationnels avec leur station de communication la plus récente
- Ex. 11 : Curseur paramétré — Fenêtres de communication d'une station donnée, avec volume total téléchargé

Palier 5 — Procédures et fonctions standalone (Socle)

- Ex. 12 : Exceptions prédéfinies — SELECT INTO sécurisé sur SATELLITE avec gestion de NO_DATA_FOUND et OTHERS
- Ex. 13 : RAISE_APPLICATION_ERROR — Validation d'une fenêtre de communication avant insertion (satellite opérationnel, station active, pas de chevauchement)
- Ex. 14 [Procédure — Socle] : afficher_statut_satellite(p_id IN) — affiche le statut, l'orbite et les instruments du satellite
- Ex. 15 [Procédure — Socle] : mettre_a_jour_statut(p_id IN, p_statut IN, p_ancien_statut OUT) — retourne l'ancien statut via OUT
- Ex. 16 [Fonction — Socle] : calculer_volume_session(p_id_fenetre IN) RETURN NUMBER — retourne le volume théorique (débit × durée)

Note socle : Les exercices 14, 15 et 16 constituent les livrables minimum de ce palier (3 sous-programmes standalone). Les exercices suivants du palier 6 sont bonus.

3.4 Palier 6 — Le package pkg_nanoOrbit ★ Niveau Excellence

> **★ Palier Niveau 2**
> Ce palier est réservé aux groupes ayant finalisé les paliers 1 à 5. Il n'est pas requis pour valider le projet mais apporte des points bonus significatifs. La SPEC doit compiler (SHOW ERRORS = 0 erreur) avant toute implémentation du BODY.

Assemblez l'ensemble des opérations métier NanoOrbit dans un package Oracle unique. Créez la SPEC en premier, vérifiez qu'elle compile (SHOW ERRORS), puis créez le BODY.

SPEC du package — Interface publique

La SPEC doit déclarer les éléments suivants :

- Type public t_stats_satellite : enregistrement contenant nb_fenetres NUMBER, volume_total NUMBER, duree_moy_secondes NUMBER
- Constantes métier : c_statut_min_fenetre (satellite doit être Opérationnel), c_duree_max_fenetre (900), c_seuil_revision (50 fenêtres)
- Procédure planifier_fenetre(p_id_satellite, p_code_station, p_datetime_debut, p_duree, p_id_fenetre OUT)
- Procédure cloturer_fenetre(p_id_fenetre, p_volume_donnees)
- Procédure affecter_satellite_mission(p_id_satellite, p_id_mission, p_role)
- Procédure mettre_en_revision(p_id_satellite)
- Fonction calculer_volume_theorique(p_id_fenetre) RETURN NUMBER
- Fonction statut_constellation RETURN VARCHAR2 — retourne un résumé textuel (ex : "8/10 satellites opérationnels, 3 missions actives")
- Fonction stats_satellite(p_id_satellite) RETURN t_stats_satellite

BODY du package — Implémentation

Implémentez chaque sous-programme du BODY. Les procédures doivent s'appuyer sur les triggers de la Phase 2 lorsque c'est pertinent (T1 déclenché par INSERT dans planifier_fenetre, T5 déclenché par UPDATE dans mettre_en_revision). La gestion des exceptions doit être exhaustive avec des codes -20xxx cohérents.

Scénario de validation du package

Rédigez un bloc de test enchaîné qui appelle les 7 sous-programmes du package dans un scénario réaliste :

- Planifier une nouvelle fenêtre de communication pour SAT-001 vers GS-KIR-01
- Clôturer la fenêtre avec un volume de données
- Affecter SAT-004 à la mission MSN-ARC-2023 avec le rôle "Satellite de relais"
- Appeler stats_satellite('SAT-001') et afficher les 3 indicateurs
- Appeler statut_constellation() et afficher le résumé

3.5 Livrables Phase 3

Réf.	Livrable	Contenu attendu
L3-A	**Script Paliers 1–5**	Exercices 1 à 16 avec résultats attendus en commentaire (SET SERVEROUTPUT ON en tête)
L3-B	**Script SPEC pkg_nanoOrbit**	SPEC compilée (SHOW ERRORS après CREATE OR REPLACE PACKAGE)
L3-C	**Script BODY pkg_nanoOrbit**	BODY compilé (SHOW ERRORS après CREATE OR REPLACE PACKAGE BODY)
L3-D	**Script de validation**	Scénario de test enchaîné du package complet avec sorties DBMS_OUTPUT commentées

PHASE 4
Exploitation avancée & Optimisation ★ Niveau 2

Vues · CTE · Fonctions analytiques · MERGE INTO · Index · EXPLAIN PLAN

Durée indicative 3 h 30	Prérequis Phases 1 à 3 terminées pkg_nanoOrbit opérationnel	SGBD Oracle 23ai

★ Phase 4 — Niveau Excellence
Cette phase est destinée aux groupes ayant finalisé les phases 1, 2 et 3. Les exercices V1 à V3 (vues simples) et les exercices 5 à 10 (CTE + sous-requêtes) constituent le socle minimum de cette phase. Les fonctions analytiques, MERGE INTO, EXPLAIN PLAN et le rapport de pilotage sont des exercices d'excellence.

4.1 Objectifs pédagogiques

- Créer et exploiter des vues simples, complexes et matérialisées sur le schéma NanoOrbit
- Structurer des requêtes complexes avec WITH … AS (CTE), y compris des CTE récursives
- Écrire des sous-requêtes scalaires, corrélées et utiliser EXISTS / NOT EXISTS
- Utiliser les fonctions analytiques OVER (PARTITION BY … ORDER BY …)
- Synchroniser des données avec MERGE INTO
- Lire un EXPLAIN PLAN et créer des index pour optimiser les requêtes critiques

4.2 Partie 1 — Vues (CREATE VIEW)

Créez les 4 vues suivantes. Pour chacune, fournissez le code et une requête SELECT de test avec le résultat attendu.

Vue	Objet	Contenu attendu
V1 — v_satellites_operationnels	Vue simple filtrée	Satellites opérationnels avec nom d'orbite, nb instruments embarqués, statut batterie
V2 — v_fenetres_detail	Vue jointure dénormalisée	Toutes les fenêtres avec noms satellite, station, centre de contrôle, durée formatée, volume
V3 — v_stats_missions	Vue avec agrégats	Par mission : nb satellites, types d'orbites représentés, volume total téléchargé
V4 — mv_volumes_mensuels	Vue matérialisée (REFRESH ON DEMAND)	Volumes téléchargés par mois, par centre de contrôle et par type de satellite

4.3 Partie 2 — CTE avec WITH … AS

- Ex. 5 — CTE simple : Top 3 des satellites ayant téléchargé le plus grand volume de données, avec nombre de fenêtres réalisées et volume moyen par passage
- Ex. 6 — CTE multiples : Analyse comparative par centre de contrôle — nombre de fenêtres ce mois, volume total, station la plus active
- Ex. 7 — CTE récursive : Hiérarchie Centre de contrôle → Station au sol → Fenêtres récentes, avec indentation visuelle (LPAD)

4.4 Partie 3 — Sous-requêtes avancées

- Ex. 8 — Sous-requête scalaire : Lister les fenêtres dont le volume téléchargé est supérieur à la moyenne générale, en affichant l'écart à la moyenne
- Ex. 9 — Sous-requête corrélée : Pour chaque satellite, récupérer sa dernière fenêtre de communication réalisée (date, station, volume)
- Ex. 10 — EXISTS / NOT EXISTS : Identifier les satellites sans aucune fenêtre de communication réalisée ET les stations n'ayant traité aucune fenêtre ce trimestre (expliquez en commentaire pourquoi une station peut être dans cette situation)

4.5 Partie 4 — Fonctions analytiques OVER

- Ex. 11 — ROW_NUMBER / RANK / DENSE_RANK : Classement des satellites par volume de données téléchargé, global et par type d'orbite (PARTITION BY type_orbite)
- Ex. 12 — LAG / LEAD : Pour chaque fenêtre de communication d'une station, comparer le volume avec la fenêtre précédente et calculer l'évolution en pourcentage
- Ex. 13 — SUM OVER : Volumes cumulés chronologiquement par centre de contrôle, avec moyenne mobile sur les 3 dernières fenêtres
- Ex. 14 — Tableau de bord constellation : Combiner RANK, SUM OVER et ROUND pour produire le rapport mensuel (rang satellite, part % du volume total, cumul, comparaison à la moyenne)

4.6 Partie 5 — MERGE INTO

- Ex. 15 — Synchroniser un lot de mises à jour de statuts de satellites reçues d'un système IoT externe : si le satellite existe, mettre à jour son statut et son orbite courante ; s'il est nouveau (cas rare), l'insérer avec un statut "En veille"
- Ex. 16 — Synchroniser les affectations de stations aux centres de contrôle depuis un fichier de configuration révisé : mettre à jour les dates d'affectation pour les associations existantes, créer les nouvelles associations

4.7 Partie 6 — Index & EXPLAIN PLAN

- Ex. 17 — Créer les index stratégiques manquants : index sur les FK de FENETRE_COM et PARTICIPATION, index sur statut de SATELLITE, index composite (statut, type_orbite) sur SATELLITE, index fonctionnel sur TRUNC(datetime_debut, 'MM') dans FENETRE_COM
- Ex. 18 — Lire et interpréter l'EXPLAIN PLAN d'une requête de reporting mensuel (JOIN sur 4 tables + GROUP BY + agrégats) — identifier les TABLE ACCESS FULL et proposer les index correctifs
- Ex. 19 — Mesurer l'impact : rendre invisible l'index sur statut (ALTER INDEX … INVISIBLE), relever le plan (TABLE ACCESS FULL attendu), rendre visible à nouveau (INDEX RANGE SCAN attendu) — documenter la différence

Rapport de pilotage intégral — Exercice de synthèse

Rédigez la requête finale combinant CTE, fonctions analytiques et vue matérialisée pour produire le tableau de bord opérationnel de NanoOrbit : rang des centres de contrôle par volume téléchargé, part % du volume total, évolution par rapport au mois précédent (LAG), et statut de chaque satellite rattaché.

4.8 Livrables Phase 4

Réf.	Livrable	Contenu attendu
L4-A	**Script Vues**	V1–V3 (CREATE VIEW) + V4 (vue matérialisée) avec SELECT de test
L4-B	**Script CTE & Sous-requêtes**	Exercices 5 à 10 commentés avec résultats attendus
L4-C	**Script Analytiques & MERGE**	Exercices 11 à 16 commentés
L4-D	**Script Index & EXPLAIN PLAN**	Exercices 17 à 19 + rapport de pilotage final, plans avant/après documentés

5. Récapitulatif de l'évaluation

Phase	Contenu	Thèmes ALTN83	Niveau 1	Niveau 2
Phase 1	**Conception & Architecture distribuée**	MCD/MLD, Répartition	**20 pts**	—
Phase 2	**Schéma Oracle & Triggers**	LDD, Triggers PL/SQL	**20 pts**	**+ 5 pts ★**
Phase 3	**PL/SQL & Package pkg_nanoOrbit**	PL/SQL, Packages	**25 pts**	**+ 15 pts ★**
Phase 4	**Exploitation avancée & Optimisation**	Optimisation, Vues	**15 pts**	**+ 25 pts ★**
TOTAL			**80 pts socle**	**+ 45 pts ★**

6. Organisation et modalités de rendu

Modalité	Détail
Groupes	Binômes ou trinômes — constitués librement en début de module
SGBD cible	Oracle 23ai — schéma NANOORBIT_ADMIN sur FREEPDB1
Rendu par phase	Un fichier .sql unique par phase, commenté, avec en-tête (noms, groupe, date, SGBD)
Nom des fichiers	GROUPE_NomA_NomB_NanoOrbit_Phase[1-4].sql + Phase1_NomA_NomB.pdf pour les livrables modélisation
Présentation orale	15 min de présentation + 10 min de questions en fin de module — démonstration live sur Oracle
Plagiat	Tout partage de code entre groupes est sanctionné par la note 0/20 et une convocation en conseil de discipline